

TECHNICAL REPORT

Automatic get Street Name from Lat Lon (Parking MJBSpot)

Executed & Prepared By:

Nur Sabrina Binti Mohd Sazali (Executive CDU)

Date: 15/10/2024

Executive Summary:

The Parking data table doesn't show street names, which is important to know where people park. The addresses(latitude, longitude) we have are only for the HOURLY product, which shows where the app owner is when they purchase the product. This doesn't tell us exactly where customers park.

Even though we don't have exact locations, we use this information to see where customers park. This helps us learn about their habits in different areas. By changing the latitude and longitude into street names, we can make sense of the data. This helps us see parking trends and better manage parking resources.

Introduction:

We collect latitude and longitude data for hourly parking purchases, but this only shows the user's approximate location, not the exact parking spot. This project will use Python to convert latitude and longitude into street names for each transaction. By doing this, we will improve our data analysis. It will help us understand user behavior better, identify busy areas, and allocate resources effectively. This geolocation improvement will give Presgo a clearer view of parking patterns.

Methodology

This project uses SQL databases, Python, and Power BI to improve parking transaction data by adding street names. Here are the key steps:

1. Data Storage and Extraction (SQL Database): We store parking data in the MBJBSPOT_REPLICATION database, specifically in two tables: StreetFromLocation and vw_PARKPROD_DAILY. A stored procedure, [dbo].[SP_ParkingHourlyByStreet], runs every 10 minutes to add new hourly parking data for analysis.

2. Geocoding with Python: We use a Python script to connect to the SQL server and find records that don't have street names. The script gets latitude and longitude data from these records and uses the Nominatim API to convert these coordinates into street names. It then updates the database with the new names. At first, to manage rate limits, the script ran every 5 minutes.

Improvement: To fix the rate limit issues, we will set up a self-hosted Nominatim server using Docker. This will let us make unlimited geocoding requests, making updates faster and more reliable.

3. Data Visualization (Power BI): Power BI will take the enriched data from StreetFromLocation for reporting and visualization. This will help us analyze parking patterns and customer behavior.

By using a self-hosted Nominatim server, we will improve how quickly we can process and update parking location data.

Technical Background:

1. Key Technical Concepts

- Geocoding: This is the process of converting latitude and longitude coordinates into human-readable location information, such as street names.
- Batch Processing: To manage and update large volumes of parking transaction data, batch processing is used.
- Automation with Scheduled Jobs: Automation plays a critical role in keeping data current and minimizing manual intervention.

2. Relevant Technologies

- SQL Database (Microsoft SQL Server)
- Python for Data Processing and Geocoding
- Nominatim (OpenStreetMap Geocoding)
- Power BI for Data Visualization

3. Industry Standards

- API Usage and Rate Limits
- ETL (Extract, Transform, Load) Process Standards
- Business Intelligence (BI) Best Practices

System Design/Process Description:

1. SQL Database

- a. Table:
 - i. [MBJBSPOT_REPLICATION].[dbo].[StreetFromLocation]
 - ii. [MBJBSPOT_REPLICATION].[dbo].[vw_PARKPROD_DAILY]
- b. Store Procedure: [dbo].[SP_ParkingHourlyByStreet]
- c. Schedule Job: Job SP_ParkingHourlyByStreet
- d. Schedule job run every 10 min

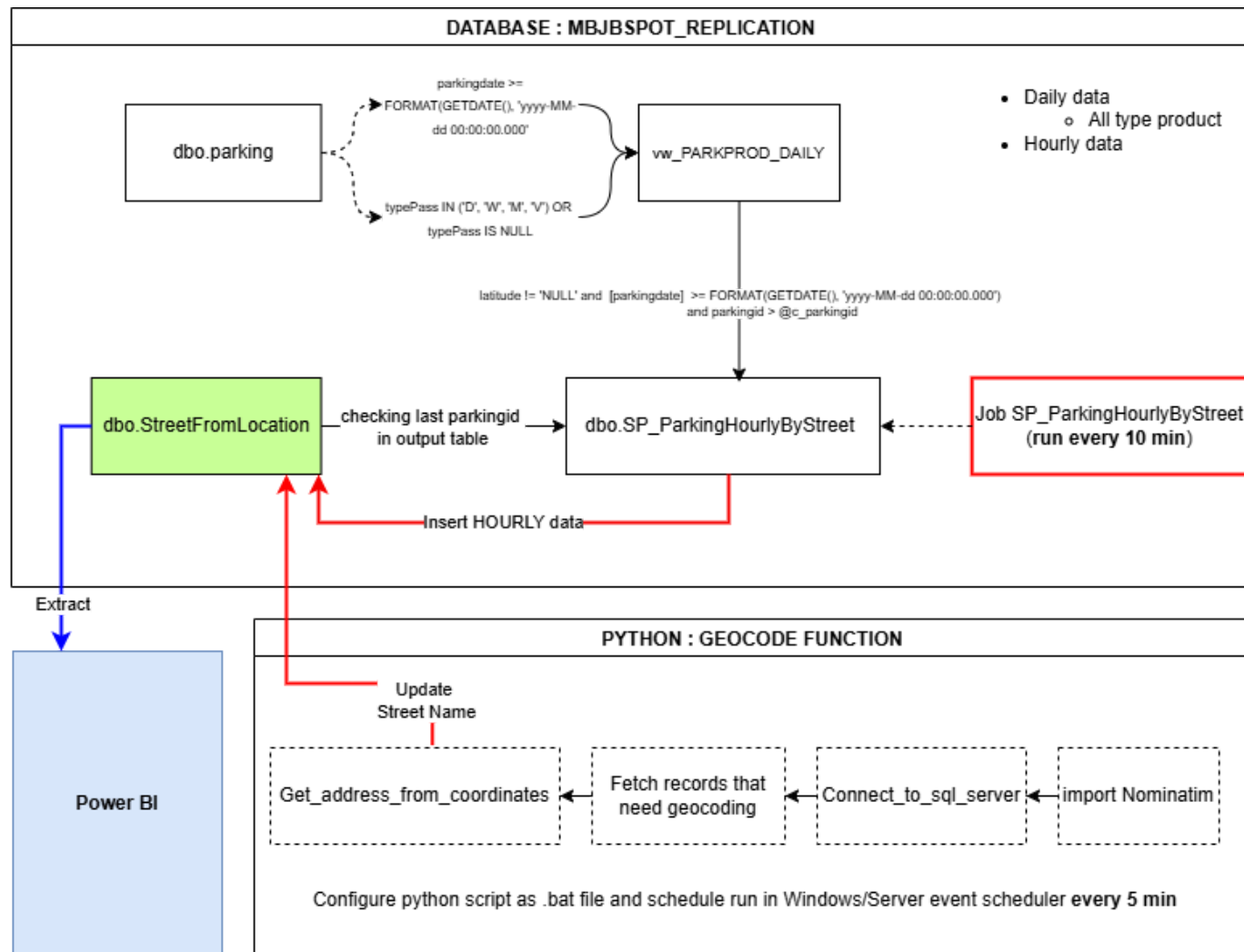
2. Python script

- a. Process:
 - i. Import Nominatim
 - ii. Connect_to_sql_server
 - iii. Fetch records that need geocoding
 - iv. Get_address_from_coordinates
 - v. Update database
- b. Configure python script as .bat file and schedule run in Windows/Server event scheduler every 5 min

3. Power BI

- a. Extract StreetFromLocation for report and visualization

Process Flow



Python Source Code:

```
import pyodbc
from geopy.geocoders import Nominatim
from geopy.exc import GeocoderTimedOut, GeocoderServiceError
import time

def connect_to_sql_server():
    """Establish connection to SQL Server"""
    conn = pyodbc.connect(
        'DRIVER={SQL Server};'
        'SERVER=192.169.0.2;'
        'DATABASE=MBJBSPOT_REPLICATION;'
        '#Trusted_Connection=yes;' # Windows authentication
        '# If using SQL authentication, use these instead:'
        'UID=sa;'
        'PWD=2363194;'
    )
    return conn

def get_address_from_coordinates(latitude, longitude, geolocator):
    """Perform reverse geocoding with retry logic"""
    max_retries = 3
    for attempt in range(max_retries):
        try:
            location = geolocator.reverse((latitude, longitude), exactly_one=True)
            if location:
                return location.raw['address']
            return None
        except (GeocoderTimedOut, GeocoderServiceError):
            if attempt == max_retries - 1:
                raise
            time.sleep(1) # Wait before retrying

def main():
    try:
        # Initialize geocoder
        geolocator = Nominatim(user_agent="my_reverse_geocoder")

        # Connect to SQL Server
```

```

conn = connect_to_sql_server()
cursor = conn.cursor()

# Fetch records that need geocoding
cursor.execute("""
    SELECT parkingid, latitude, longitude
    FROM [MBJBSPOT_REPLICATION].[dbo].[StreetFromLocation]
    WHERE loc_add = "
    AND latitude IS NOT NULL
    AND longitude IS NOT NULL
    """)

records = cursor.fetchall()

# Process each record
for record in records:
    parkingid, lat, lon = record
    try:
        # Get address from coordinates
        address_data = get_address_from_coordinates(lat, lon, geolocator)

        if address_data:
            # Extract relevant address components
            street = address_data.get('road', "")
            house_number = address_data.get('house_number', "")
            city = address_data.get('city', address_data.get('town', ""))
            state = address_data.get('state', "")

            # Construct full address and street
            full_address = f"{house_number} {street}, {city}, {state}".strip()
            street_name = f"{street}".strip()

            # Update database
            cursor.execute("""
                UPDATE [MBJBSPOT_REPLICATION].[dbo].[StreetFromLocation]
                SET
                    loc_add = ?,
                    loc_street = ?
                WHERE parkingid = ?
                """, (full_address, street_name, parkingid))

            conn.commit()
            print(f"Updated record {parkingid} with address: {full_address}")

```


Add delay to respect API rate limits

time.sleep(1)

except Exception as e:

print(f"Error processing record {parkingid}: {str(e)}")

continue

except Exception as e:

print(f"An error occurred: {str(e)}")

finally:

if 'conn' in locals():

conn.close()

if __name__ == "__main__":

main()